Titre du cours : Mise en place d'un écosystème d'IA
Code officiel : 420-A57-BB
Professeur : Dr Komi SODOKE

MODULE 04

Utiliser les technologies de virtualisation dans le contexte de l'IA

Spécialisation technique en Intelligence Artificielle

Cloud & IaaS

Docker & Conteneurs

Big Data

PLAN DU COURS

Objectif terminal : Utiliser les technologies de virtualisation dans le contexte de l'IA

I	**Introduction** Pourquoi la virtualisation est incontournable en IA
II	**Fournisseurs d'infrastructure cloud** AWS · GCP · Azure — Services GPU, tarification, cas d'usage
III	**Niveaux de virtualisation** Machine virtuelle · Conteneurisation · Serverless — Définitions et comparaison
IV	**Outils de conteneurisation : Docker** Images · Conteneurs · Dockerfile · Docker Compose · Docker Hub
V	**Données massives et virtualisation** Hadoop · Spark · Cloudera · Data Lakes
VI	**Questions de compréhension**
VII	**Conclusion & Synthèse**

I

Introduction

Du logiciel local au cloud : pourquoi virtualiser en IA ?



I — Rappel : où en sommes-nous ?

Section I

Vous avez configuré une machine IA complète (Modules 2 et 3). Mais dans le monde réel, une seule machine ne suffit pas toujours.



Modules 2-3 (faits)

1 machine locale

OS, pilotes GPU, Python, PyTorch configurés sur VOTRE machine.



Module 4 (maintenant)

Virtualisation & Cloud

Utiliser des machines distantes, des conteneurs et le cloud pour scaler l'IA.



Module 5 (à venir)

Maintenance

Monitoring, alertes et maintenance de l'infrastructure.

I — Qu'est-ce que la virtualisation ?

Section I

La virtualisation est une technologie qui permet de créer des versions « virtuelles » (logicielles) de ressources physiques : serveurs, stockage, réseaux. Au lieu d'utiliser directement le matériel, on crée des couches d'abstraction qui simulent le matériel. Résultat : plusieurs « machines virtuelles » peuvent tourner sur un seul serveur physique.

Analogie : Un immeuble d'appartements. Au lieu de construire une maison pour chaque famille (= un serveur physique par projet), on construit un immeuble (= un serveur puissant) avec plusieurs appartements isolés (= machines virtuelles). Chaque famille a son propre espace, mais partage la structure.

Pourquoi virtualiser en IA ?

Coût	Un GPU A100 coûte 15 000 $. En cloud, vous le louez à 3 $/heure. Pas d'investissement initial.
Scalabilité	Besoin de 8 GPU pendant 2 jours ? Le cloud les fournit en 5 minutes. Puis vous les libérez.
Reproductibilité	Docker garantit que votre code fonctionne identiquement sur votre PC, sur le serveur du labo et en production.

I — Les problèmes sans virtualisation

Section I

Sans virtualisation, un projet IA se heurte rapidement à ces obstacles :

"Ça marche sur ma machine"

Votre code fonctionne sur votre PC mais pas sur celui de votre collègue. Différences de versions Python, bibliothèques, OS. Docker résout ce problème : le conteneur embarque TOUT l'environnement.

GPU trop cher à acheter

Un serveur 4× A100 coûte ~50 000 $. Pour une startup ou un étudiant, c'est impossible. Le cloud permet de louer la puissance GPU à l'heure, sans investissement.

Matériel sous-utilisé

Votre GPU coûteux est inactif 80% du temps (nuits, weekends). La virtualisation permet de partager une machine entre plusieurs projets ou utilisateurs.

Données trop volumineuses

Un dataset de 10 To ne tient pas sur votre laptop. Les plateformes Big Data (Hadoop, Spark) distribuent les données sur plusieurs machines virtuelles.

II

Fournisseurs d'infrastructure cloud (IaaS)

AWS · Google Cloud · Microsoft Azure



II — Qu'est-ce que le cloud computing ?

Section II

Le cloud computing (infonuagique) consiste à louer des ressources informatiques (serveurs, stockage, GPU) à distance via Internet, au lieu de les acheter et les gérer soi-même. Vous payez uniquement ce que vous utilisez, comme l'électricité ou l'eau. Les serveurs sont dans des data centers gérés par des entreprises comme Amazon, Google ou Microsoft.

Analogie : Le cloud, c'est comme un hôtel. Au lieu d'acheter une maison (serveur physique), vous louez une chambre (machine virtuelle) pour la durée dont vous avez besoin. Le ménage (maintenance) est inclus.

Les 3 modèles de service cloud :

IaaS

Infrastructure as a Service

Vous louez le matériel (serveurs, GPU, stockage). Vous gérez tout le reste (OS, logiciels). Ex : EC2 (AWS), Compute Engine (GCP).

Louer un terrain nu — vous construisez

PaaS

Platform as a Service

Le fournisseur gère le matériel ET l'OS. Vous ne gérez que votre code. Ex : Google Colab, SageMaker (AWS).

Louer un atelier équipé — vous travaillez

SaaS

Software as a Service

Tout est géré. Vous utilisez un logiciel prêt à l'emploi via le navigateur. Ex : Gmail, Google Docs, ChatGPT.

Aller au restaurant — vous mangez

II.A — Amazon Web Services (AWS)

Section II

Amazon Web Services (AWS)

Leader mondial du cloud (33% de parts de marché)

EC2 (Elastic Compute Cloud)	Machines virtuelles à la demande. Instances GPU : p4d.24xlarge (8× A100, 320 Go VRAM) pour l'entraînement massif. Instances g5 (A10G) pour l'inférence.
S3 (Simple Storage Service)	Stockage illimité de fichiers (datasets, modèles). 0.023 $/Go/mois. Accessible depuis n'importe quelle machine.
SageMaker	Plateforme ML complète : notebooks Jupyter managés, entraînement distribué, déploiement en un clic. Pré-configuré avec PyTorch/TensorFlow.
Lambda	Serverless : exécuter du code sans gérer de serveur. Idéal pour l'inférence légère (API de classification).

Tarif GPU indicatif : p3.2xlarge (1× V100, 16 Go) ≈ 3.06 $/h · p4d.24xlarge (8× A100) ≈ 32.77 $/h · g5.xlarge (A10G) ≈ 1.01 $/h. Instances Spot : jusqu'à -90% si flexibilité horaire.

II.B — Google Cloud Platform (GCP)

Section II

Google Cloud Platform (GCP)

Le cloud de Google — spécialisé IA et Big Data

Compute Engine	Machines virtuelles avec GPU (A100, T4, L4). Equivalent d'EC2. Tarification à la seconde (pas à l'heure).
TPU (Tensor Processing Unit)	Processeurs conçus par Google spécifiquement pour le ML. TPU v4 : 275 TFLOPS. Utilisés pour entraîner les modèles Google (BERT, PaLM, Gemini).
Vertex AI	Plateforme ML managée (équivalent SageMaker). AutoML pour les non-experts : créer un modèle sans coder.
Google Colab	Notebooks Jupyter gratuits avec GPU T4 (15 Go VRAM). Déjà utilisé dans le Module 3. C'est un service GCP.
BigQuery	Entrepôt de données serverless. Analyser des téraoctets en quelques secondes avec du SQL. Idéal pour le pré-traitement de données massives.

Avantage GCP pour l'IA : TPU exclusif, Google Colab gratuit, BigQuery pour le Big Data, 300 $ de crédit gratuit pour les nouveaux comptes. TensorFlow est développé par Google → intégration native.

II.C — Microsoft Azure

Section II

Microsoft Azure

N°2 mondial — intégration entreprise et OpenAI

Azure VMs (GPU)	Machines virtuelles avec GPU NVIDIA (NC-series: T4, ND-series: A100). Intégration Active Directory pour les entreprises.
Azure ML Studio	Plateforme ML visuelle (drag & drop) + notebooks. Équivalent de SageMaker/Vertex AI.
Azure OpenAI Service	Accès exclusif aux modèles OpenAI (GPT-4, DALL-E, Whisper) via API Azure. Unique à Azure.
Azure Blob Storage	Stockage objet (équivalent S3). Intégration native avec les outils Microsoft (Office 365, Power BI).

Comparaison rapide des 3 fournisseurs :

II.D — Comparaison AWS vs GCP vs Azure

Section II

Critère	AWS	Google Cloud	Azure
Part de marché	33% (leader)	11%	22%
GPU disponibles	V100, A100, H100, A10G	T4, A100, L4, TPU	T4, A100, H100
Spécialité IA	SageMaker, large écosystème	TPU, Vertex AI, Colab	OpenAI (GPT-4), Azure ML
Tarification GPU	À l'heure (Spot -90%)	À la seconde	À la minute
Crédit gratuit	750h EC2/mois (12 mois)	300 $ (90 jours)	200 $ (30 jours)
Stockage	S3 (leader)	Cloud Storage	Blob Storage
Big Data	EMR, Athena, Redshift	BigQuery, Dataproc	Synapse, HDInsight
Force	Maturité, écosystème	IA/ML natif, TPU	Entreprises, OpenAI

Recommandation pour un étudiant : commencez par Google Colab (gratuit, GPU T4, zéro config). Pour un projet sérieux, utilisez les crédits gratuits de chaque fournisseur : GCP (300 $), AWS (12 mois de tier gratuit), Azure (200 $). Pour une entreprise : le choix dépend de l'écosystème existant (Office 365 → Azure, Android/TF → GCP, e-commerce → AWS).

III

Niveaux de virtualisation

Machine virtuelle · Conteneur · Serverless



III.A — Machine virtuelle (VM) : définition

Section III

Une machine virtuelle (VM — Virtual Machine) est un ordinateur complet simulé par logiciel à l'intérieur d'un autre ordinateur. Elle possède son propre système d'exploitation, sa propre mémoire et son propre espace disque, mais tout cela est virtualisé — c'est-à-dire simulé par un programme appelé hyperviseur qui tourne sur la machine physique (l'hôte).

Analogie : Une VM, c'est comme un simulateur de vol. Vous êtes « dans un avion » avec tous les instruments, mais en réalité vous êtes assis dans une salle. L'hyperviseur est le simulateur qui crée cette illusion.

Concepts clés :

Hyperviseur	Le logiciel qui crée et gère les VMs. Type 1 (bare-metal) : tourne directement sur le matériel (VMware ESXi, KVM). Type 2 : tourne dans un OS (VirtualBox, VMware Workstation).
Guest OS	Le système d'exploitation qui tourne DANS la VM. Peut être différent de l'hôte : un Linux dans un Windows, ou inversement.
Snapshot	Photo instantanée de l'état complet de la VM. Permet de revenir à un état précédent en cas de problème. Très utile pour tester des installations risquées.

III.A — VM : avantages et limites en IA

Section III

Avantages

- Isolation complète : chaque VM est indépendante
- Sécurité : si une VM est compromise, les autres sont protégées
- OS différents sur la même machine (Linux + Windows)
- Snapshots : retour arrière en un clic
- GPU passthrough : attribuer un GPU physique à une VM

Limites

- Lourdes : chaque VM embarque un OS complet (plusieurs Go)
- Démarrage lent (30 secondes à quelques minutes)
- Gaspillage de ressources (RAM/CPU réservées même si inutilisées)
- Pas idéal pour le déploiement rapide de microservices
- Le GPU passthrough est complexe à configurer

Quand utiliser une VM en IA ?

Les instances cloud (EC2, Compute Engine) sont des VMs. Chaque fois que vous « louez un GPU dans le cloud », vous utilisez une VM avec GPU passthrough. Les VMs sont idéales pour : l'entraînement de modèles lourds en cloud, les environnements de développement isolés, et les serveurs de notebooks partagés (JupyterHub).

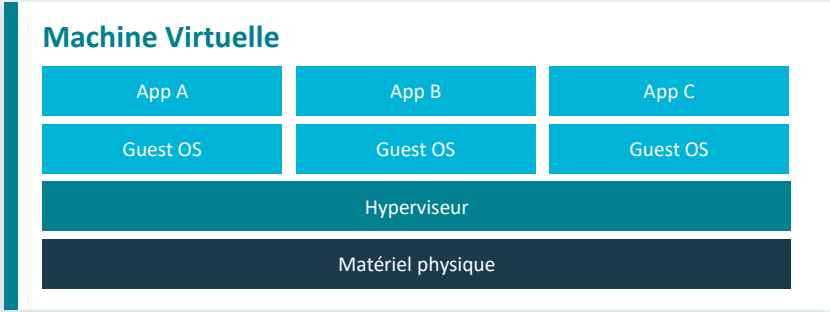
III.B — Conteneurisation : définition

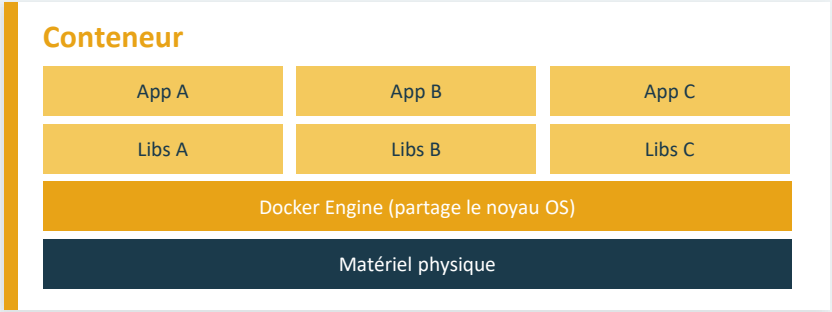
Section III

Un conteneur est un environnement d'exécution léger et isolé qui embarque une application avec TOUTES ses dépendances (bibliothèques, configuration, fichiers). Contrairement à une VM, un conteneur ne contient PAS de système d'exploitation complet — il partage le noyau (kernel) de la machine hôte. Résultat : démarrage en quelques secondes, taille en mégaoctets au lieu de gigaoctets.

Analogie : Si une VM est un immeuble complet (fondations, murs, toit, plomberie), un conteneur est un container maritime standardisé. Il contient tout ce dont vous avez besoin, il est portable (bateau, train, camion) et il se pose/dépose en quelques secondes.

VM vs Conteneur : la différence clé





III.C — Serverless (sans serveur)

Section III

Le serverless est un modèle cloud où vous ne gérez AUCUNE infrastructure. Vous écrivez une fonction, vous la déployez, et le fournisseur cloud l'exécute automatiquement quand elle est appelée. Pas de serveur à provisionner, pas d'OS à maintenir, pas de scaling à gérer. Vous payez uniquement le temps d'exécution réel de votre code (à la milliseconde).

Analogie : Le serverless, c'est comme un taxi. Vous ne possédez pas de voiture, vous n'avez pas de garage, pas d'assurance. Vous appelez un taxi quand vous en avez besoin et vous payez la course. Le reste du temps, vous ne payez rien.

Services serverless pour l'IA :

AWS Lambda	Fonctions exécutées à la demande. Max 15 min d'exécution, 10 Go de RAM. Idéal pour des API d'inférence légères (classification d'image, analyse de texte).
Google Cloud Functions	Équivalent Lambda chez Google. Déclenchement automatique sur événements (nouveau fichier dans Storage, requête HTTP).
Azure Functions	Équivalent Microsoft. Intégration native avec les services Azure (Cosmos DB, Event Grid).

Limite pour l'IA : le serverless n'a PAS de GPU et a un temps d'exécution limité (15 min). Impossible pour l'entraînement. Utilisable uniquement pour l'inférence légère.

III.D — Comparaison : VM vs Conteneur vs Serverless

Section III

Critère	Machine Virtuelle	Conteneur (Docker)	Serverless
Contient un OS ?	Oui (OS complet)	Non (partage le noyau)	Non (invisible)
Taille	Go (2-20 Go)	Mo (50-500 Mo)	Ko (votre code seul)
Démarrage	30s - 5 min	< 5 secondes	< 1 seconde
Isolation	Complète (matériel simulé)	Processus isolé	Fonction isolée
GPU	Oui (passthrough)	Oui (nvidia-docker)	Non
Idéal pour	Entraînement cloud, serveur dédié	Déploiement, portabilité	API légère, événements
Exemple IA	EC2 + GPU pour entraîner	Docker + PyTorch pour déployer	Lambda pour inférence
Coût	Par heure/mois	Gratuit (outil local)	Par milliseconde



En pratique en IA : on combine les 3 niveaux. Étape 1 : développer dans un conteneur Docker (reproductibilité). Étape 2 : lancer le conteneur sur une VM cloud avec GPU (entraînement). Étape 3 : déployer le modèle entraîné en serverless (inférence à faible coût). C'est le workflow standard de MLOps.

IV

Docker : l'outil de conteneurisation

Images · Conteneurs · Dockerfile · Docker Compose



IV.A — Docker : qu'est-ce que c'est ?

Section IV

Docker est un outil open-source qui permet de créer, distribuer et exécuter des conteneurs. Créé en 2013, il est devenu le standard de l'industrie pour empaqueter des applications avec toutes leurs dépendances. En IA, Docker garantit que votre code + vos bibliothèques + votre configuration fonctionnent de manière identique partout : sur votre laptop, le serveur du labo, et en production.

Pourquoi Docker a révolutionné l'IA : avant Docker, installer PyTorch + CUDA + cuDNN + toutes les dépendances prenait des heures et échouait souvent. Avec Docker, vous téléchargez une image pré-configurée et tout fonctionne en 30 secondes.

Les 3 concepts fondamentaux de Docker :

Image Docker — *= La recette de cuisine*

Un « modèle » (template) en lecture seule qui contient l'OS minimal, les bibliothèques et la configuration. C'est comme un plan de construction. On ne modifie pas une image — on en crée une nouvelle.

Conteneur Docker — *= Le plat préparé (un par table)*

Une instance en cours d'exécution d'une image. C'est le « plat cuisiné » à partir de la recette. On peut créer plusieurs conteneurs à partir de la même image. Chaque conteneur est isolé.

Dockerfile — *= Le livre de recettes*

Un fichier texte qui décrit étape par étape comment construire une image. C'est la « recette écrite » : quel OS de base, quelles bibliothèques installer, quel code copier.

IV.B — Le Dockerfile expliqué ligne par ligne

Section IV

Voici un Dockerfile typique pour un projet IA avec PyTorch :

```
# 1. Image de base : NVIDIA CUDA + Ubuntu
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04

# 2. Installer Python
RUN apt-get update && \
    apt-get install -y python3 python3-pip && \
    rm -rf /var/lib/apt/lists/*

# 3. Copier le fichier des dépendances
COPY requirements.txt /app/requirements.txt

# 4. Installer les bibliothèques Python
RUN pip install -r /app/requirements.txt

# 5. Copier le code du projet
COPY . /app
WORKDIR /app

# 6. Commande de démarrage
CMD ["python3", "train.py"]
```

Explication

FROM

Image de base. nvidia/cuda inclut CUDA et le driver GPU.

RUN

Exécute une commande pendant la construction de l'image.

COPY

Copie des fichiers depuis votre PC vers l'image.

WORKDIR

Définit le dossier de travail dans le conteneur.

CMD

Commande exécutée au démarrage du conteneur.

Commandes pour construire et lancer :

```
$ docker build -t mon-projet-ia .        # Construire l'image
$ docker run --gpus all mon-projet-ia    # Lancer avec GPU
```

IV.C — Docker Compose : orchestrer plusieurs conteneurs

Section IV

Docker Compose est un outil qui permet de définir et lancer plusieurs conteneurs en même temps avec un seul fichier (docker-compose.yml). En IA, un projet typique a besoin de : un conteneur pour l'entraînement, un pour l'API de prédiction, un pour la base de données, un pour le monitoring. Compose les orchestre tous.

```
# docker-compose.yml
version: '3.8'
services:
  training:
    build: .
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]
    volumes:
      - ./data:/app/data

  api:
    image: python:3.11-slim
    ports:
      - "8000:8000"
    command: python api.py

  db:
    image:
postgres:15
    environment:
      POSTGRES_DB:
models_db
```

Ce fichier crée :

`training`

Conteneur GPU pour entraîner le modèle. Monte le dossier data pour accéder aux datasets.

`api`

Conteneur léger qui sert les prédictions via HTTP (port 8000).

`db`

Base de données PostgreSQL pour stocker les métadonnées des modèles.

Tout lancer en une commande :

```
$ docker compose up -d    # Lance les 3 conteneurs en arrière-plan
```

IV.D — Docker Hub et images pré-configurées IA

Section IV

Docker Hub (hub.docker.com) est le « magasin » public d'images Docker. Des milliers d'images sont disponibles gratuitement, y compris des images pré-configurées avec CUDA, PyTorch, TensorFlow, etc. Au lieu de tout installer manuellement, vous téléchargez une image prête à l'emploi en une commande.

Images IA populaires sur Docker Hub :

`nvidia/cuda:12.1.0-runtime-ubuntu22.04`

Image de base NVIDIA avec CUDA. Point de départ pour tout projet GPU.

`pytorch/pytorch:2.1.0-cuda12.1-cudnn8-runtime`

PyTorch pré-installé avec CUDA et cuDNN. Prêt à entraîner immédiatement.

`tensorflow/tensorflow:latest-gpu`

TensorFlow avec support GPU. Inclut Keras et toutes les dépendances.

`huggingface/transformers-pytorch-gpu`

HuggingFace Transformers + PyTorch + GPU. Pour le NLP et les LLM.

NVIDIA NGC (ngc.nvidia.com) propose des images Docker optimisées et testées par NVIDIA pour chaque framework IA. Elles offrent souvent 20-30% de performances en plus que les images standard.

V

Données massives et virtualisation

Hadoop · Spark · Cloudera · Data Lakes



V.A — Apache Hadoop

Section V

Hadoop est un framework open-source pour le stockage et le traitement distribué de données massives. Créé par Yahoo en 2006 (inspiré des travaux de Google), il permet de répartir un dataset sur des centaines de machines et de le traiter en parallèle. C'est la technologie qui a lancé l'ère du Big Data.

Les 3 composantes de Hadoop :

HDFS (Hadoop Distributed File System)

Système de fichiers distribué. Stocke les données en blocs de 128 Mo répartis sur plusieurs machines. Chaque bloc est répliqué 3 fois pour la tolérance aux pannes. Idéal pour les fichiers volumineux (datasets, logs).

MapReduce

Moteur de calcul distribué. Divise un traitement en deux phases : Map (transformer) et Reduce (agréger). Permet de traiter des téraoctets en quelques heures en utilisant toutes les machines du cluster.

YARN (Yet Another Resource Negotiator)

Gestionnaire de ressources du cluster. Distribue les tâches (CPU, RAM) entre les applications. Permet à MapReduce, Spark et d'autres moteurs de

Hadoop est moins utilisé directement aujourd'hui (remplacé par Spark pour le calcul), mais HDFS reste le standard de stockage distribué dans de nombreuses entreprises.

V.B — Pourquoi le Big Data en IA ?

Section V

Le Big Data désigne des ensembles de données tellement volumineux, rapides ou complexes qu'ils ne peuvent pas être traités par un seul ordinateur avec des outils classiques. En IA, les datasets modernes sont massifs : ImageNet (150 Go), Common Crawl (80+ To), les données d'entraînement de GPT-4 (des pétaoctets). Ces données doivent être stockées, traitées et analysées de manière distribuée — sur plusieurs machines en parallèle.

Les 3 V du Big Data :

Volume	Quantité massive : téraoctets à pétaoctets. Un modèle de langage s'entraîne sur des milliards de phrases.	*Dataset: 1 To d'images médicales*
Vélocité	Vitesse d'arrivée des données. En temps réel : logs de serveur, données IoT, flux vidéo.	*10 000 images/seconde d'une caméra*
Variété	Types de données différents : textes, images, vidéos, sons, données tabulaires. Pas que des chiffres.	*Dossier patient = texte + images + labos*

V.C — Apache Spark

Section V

Apache Spark est un moteur de traitement de données massives ultra-rapide, créé à l'UC Berkeley en 2014. Il remplace MapReduce de Hadoop en effectuant les calculs en mémoire vive (RAM) au lieu du disque dur. Résultat : Spark est jusqu'à 100× plus rapide que MapReduce pour certaines tâches. C'est aujourd'hui le standard pour le traitement Big Data en IA.

Les modules de Spark :

Spark SQL	Requêtes SQL sur des données massives. Interface DataFrame similaire à Pandas mais distribué sur le cluster.
Spark MLlib	Bibliothèque de Machine Learning distribué. Régression, classification, clustering sur des datasets trop grands pour Scikit-learn.
Spark Streaming	Traitement de données en temps réel (flux continus). Idéal pour les données IoT, les logs de serveur.
PySpark	L'interface Python de Spark. Syntaxe similaire à Pandas mais les calculs sont distribués automatiquement sur le cluster.

Spark vs Hadoop MapReduce : Spark travaille en RAM (rapide), MapReduce sur disque (lent). Spark supporte Python (PySpark), MapReduce est principalement Java. Spark a MLlib pour le ML, MapReduce n'a pas d'équivalent intégré. En pratique, Spark utilise souvent HDFS (Hadoop) pour le stockage.

V.D — Cloudera et les Data Lakes

Section V

Cloudera

La distribution Hadoop/Spark « tout-en-un »

- Distribution commerciale de Hadoop + Spark
- Interface graphique pour gérer le cluster
- Sécurité, gouvernance et monitoring intégrés
- Utilisé par les grandes entreprises (banques, santé)
- Version gratuite : CDP Public Cloud (essai)

Data Lake

Stocker TOUTES les données brutes au même endroit

- Stockage de données brutes, non structurées
- Textes, images, vidéos, CSV, JSON, Parquet...
- On stocke d'abord, on structure après (schema-on-read)
- Implémenté avec HDFS, S3, Azure Data Lake
- Base de l'architecture data moderne en IA

Data Lake vs Data Warehouse :

	Data Lake	Data Warehouse
Données	Brutes, non structurées	Structurées, nettoyées
Format	Tout format (JSON, images, CSV)	Tableaux relationnels (SQL)
Usage	Exploration, ML, IA	Reporting, BI, dashboards
Stockage	HDFS, S3, Azure Data Lake	BigQuery, Redshift, Snowflake
Coût	Bas (stockage brut)	Élevé (données transformées)

VI

Questions de compréhension

Testez vos connaissances sur la virtualisation



VI — Questions de compréhension

Quiz

1. Qu'est-ce que la virtualisation ? Donnez une analogie simple.
2. Quelle est la différence entre IaaS, PaaS et SaaS ? Donnez un exemple de chaque pour l'IA.
3. Comparez AWS, GCP et Azure : quelle est la spécialité IA de chacun ?
4. Quelle est la différence fondamentale entre une VM et un conteneur Docker ?
5. Qu'est-ce qu'un Dockerfile ? Expliquez les instructions FROM, RUN, COPY, CMD.
6. Pourquoi Docker est-il devenu indispensable en IA ? Quel problème résout-il ?
7. Qu'est-ce que le serverless ? Peut-on entraîner un modèle de Deep Learning en serverless ? Pourquoi ?
8. Comparez Hadoop MapReduce et Spark. Lequel est le plus rapide et pourquoi ?
9. Qu'est-ce qu'un Data Lake ? En quoi est-il différent d'un Data Warehouse ?
10. Décrivez le workflow MLOps complet : développement → entraînement → déploiement en utilisant Docker et le cloud.

VI — Réponses aux questions (1-5)

Quiz

1 — Virtualisation ?

Technologie qui crée des versions logicielles de ressources physiques (serveurs, stockage). Analogie : un immeuble d'appartements — un seul bâtiment physique, plusieurs espaces isolés. En IA, elle permet de louer du GPU à la demande au lieu d'acheter du matériel.

2 — IaaS / PaaS / SaaS ?

IaaS = louer le matériel (ex : EC2 avec GPU → vous gérez l'OS et PyTorch). PaaS = plateforme prête (ex : Google Colab → vous gérez uniquement votre code Python). SaaS = logiciel prêt à l'emploi (ex : ChatGPT → vous utilisez directement sans rien installer).

3 — AWS vs GCP vs Azure ?

AWS : leader (33%), SageMaker, écosystème le plus large. GCP : spécialisé IA (TPU exclusif, Colab gratuit, Vertex AI, BigQuery). Azure : intégration entreprise (Office 365) + accès exclusif à OpenAI (GPT-4). Pour un étudiant → GCP (Colab gratuit). Pour une entreprise → dépend de l'écosystème existant.

4 — VM vs Conteneur ?

VM = ordinateur complet simulé avec son propre OS (lourd, Go, démarrage lent). Conteneur = environnement léger qui partage le noyau de l'hôte (léger, Mo, démarrage en secondes). La VM isole au niveau matériel, le conteneur isole au niveau processus. En IA : VM pour l'entraînement cloud, conteneur Docker pour le déploiement.

5 — Dockerfile ?

Fichier texte qui décrit comment construire une image Docker, étape par étape. FROM = image de base (ex : nvidia/cuda). RUN = exécuter une commande (ex : pip install pytorch). COPY = copier des fichiers du PC vers l'image. CMD = commande lancée au démarrage du conteneur (ex : python train.py).

VI — Réponses aux questions (6-10)

Quiz

6 **Pourquoi Docker en IA ?**

Docker résout le problème « ça marche sur ma machine ». Il empaquette le code + toutes les dépendances (Python, PyTorch, CUDA, bibliothèques) dans un conteneur identique partout. Avant Docker : installer PyTorch+CUDA prenait des heures et échouait souvent. Avec Docker : une image pré-configurée fonctionne en 30 secondes.

7 **Serverless et Deep Learning ?**

Le serverless (Lambda, Cloud Functions) n'a PAS de GPU et a un temps d'exécution limité (15 min max). Impossible pour l'entraînement DL (qui prend des heures/jours). Utilisable uniquement pour l'inférence légère : recevoir une image, la classifier avec un petit modèle pré-entraîné, retourner le résultat. Paiement à la milliseconde = très économique.

8 **Hadoop vs Spark ?**

Hadoop MapReduce travaille sur disque dur (lent, écrit les résultats intermédiaires sur disque). Spark travaille en mémoire RAM (jusqu'à 100× plus rapide, garde les données en mémoire entre les itérations). Spark a PySpark (Python) et MLlib (ML distribué). Hadoop est principalement Java. Spark est le standard actuel pour le Big Data en IA.

9 **Data Lake ?**

Stockage centralisé de données brutes dans tous les formats (textes, images, vidéos, CSV, JSON). Contrairement à un Data Warehouse (données structurées en tableaux SQL), un Data Lake stocke d'abord et structure après (schema-on-read). En IA, c'est essentiel car les données ML sont souvent non structurées. Implémenté avec HDFS, S3 ou Azure Data Lake.

10 **Workflow MLOps complet ?**

1) Développer dans un notebook Jupyter. 2) Créer un Dockerfile (PyTorch + CUDA + code). 3) Construire l'image et la pousser sur Docker Hub. 4) Louer une VM GPU cloud (EC2 ou Compute Engine). 5) Tirer l'image Docker et lancer l'entraînement. 6) Sauvegarder le modèle sur S3/GCS. 7) Déployer l'API d'inférence dans un conteneur séparé (ou serverless pour les petits modèles).

CONCLUSION



Le cloud (AWS, GCP, Azure) permet de louer de la puissance GPU sans investissement. GCP offre Colab gratuit et les TPU exclusifs.



3 niveaux de virtualisation : VM (isolation totale), Conteneur (léger et portable), Serverless (payer à l'usage). En IA, on combine les 3.



Docker est devenu indispensable : il garantit la reproductibilité ("ça marche partout") et simplifie le déploiement des modèles IA.



Pour les données massives : Hadoop (stockage distribué), Spark (calcul en mémoire, 100× plus rapide), PySpark pour l'interface Python.



Le workflow MLOps moderne : développer dans Docker → entraîner sur le cloud avec GPU → déployer en serverless ou conteneur.

Prochain module : Contrôler et entretenir l'infrastructure matérielle dans le contexte de l'IA (Monitoring, Alertes, Maintenance)

Glossaire — Termes clés

Référence

Terme	Définition
`Cloud computing`	Location de ressources informatiques (serveurs, GPU, stockage) via Internet, à la demande.
`IaaS`	Infrastructure as a Service. Location du matériel. Vous gérez l'OS et les logiciels.
`VM`	Machine Virtuelle. Ordinateur simulé par logiciel avec son propre OS.
`Hyperviseur`	Logiciel qui crée et gère les machines virtuelles (ex: KVM, VMware).
`Conteneur`	Environnement d'exécution léger qui partage le noyau de l'hôte. Plus léger qu'une VM.
`Docker`	Outil standard pour créer et exécuter des conteneurs.
`Dockerfile`	Fichier texte décrivant comment construire une image Docker, étape par étape.
`Docker Compose`	Outil pour orchestrer plusieurs conteneurs avec un seul fichier YAML.
`Serverless`	Modèle cloud où le fournisseur gère toute l'infrastructure. Paiement à l'exécution.
`Hadoop`	Framework de stockage (HDFS) et traitement distribué (MapReduce) de données massives.
`Spark`	Moteur de traitement Big Data en mémoire, jusqu'à 100× plus rapide que MapReduce.
`Data Lake`	Stockage centralisé de données brutes (tous formats) pour l'analyse et le ML.